Carbonique Hub installation on Arbutus

A step by step guide for the existing Ubuntu virtual machine

Ubuntu 24.04 | PostgreSQL | Gunicorn | HTTPS
21 October 2026

Follow the checks in order before allowing team writes. This guide covers a fresh installation or migration of your existing records, shared VM operation, phones, backups and troubleshooting. Use the Markdown companion to copy long commands without visual line wraps.

Contents

1 Prepare the deployment . . . 2

2 Connect and inspect the existing server . . . 2

3 Protect tower data and plan storage . . . 3

4 Tighten Arbutus network access . . . 3

5 Check the guest firewall without losing access . . . 4

6 Install the runtime and application account . . . 5

7 Install the reviewed source . . . 5

8 Create a dedicated PostgreSQL database . . . 5

9 Configure private production settings . . . 6

10 Initialize or migrate your current records . . . 7

11 Run Gunicorn as a managed service . . . 8

12 Add a hostname to an existing nginx server . . . 8

13 Apache alternative when Apache already owns HTTPS . . . 10

14 Configure tower status privately . . . 10

15 Verify the application and phone experience . . . 11

16 Back up and practice a restore . . . 11

17 Update and recover a release . . . 12

18 Troubleshoot common problems . . . 12

19 Plan future tower data plots . . . 13

20 Documentation and handover . . . 14

This guide installs Carbonique Hub on an existing Ubuntu 24.04 virtual machine in the Digital Research Alliance of Canada Arbutus cloud. It uses PostgreSQL, Gunicorn, systemd and an HTTPS reverse proxy. Docker is not required. Follow the steps in order, run commands individually, and stop when a check fails.

The supplied VM screenshots show 8 vCPUs, 12 GB RAM, a 20 GB system disk, approximately 14 GB free on the root filesystem, and an existing `/Data` NFS mount. They also show ports 80 and 443 already in use and broad inbound network rules. Recheck these observations before installation. This guide does not change the VM, its tower files, or its cloud configuration by itself.

1 Prepare the deployment

Use an approved maintenance window. Have the VM administrator available, an SSH key that already works, sudo access, Arbutus project access and the instance console available as a recovery route. The existing Alliance account and VM can be reused; another account application or floating IP is not needed.

Fill in these values privately. Example hostnames and CIDRs below are placeholders, not working addresses. Do not commit actual credentials or private network configuration to GitHub.

Value	Example or purpose
Public hostname	`carbonique.example.org`, a dedicated hostname serving the app at `/`
Floating IP	The existing public IP associated with this instance
SSH user and key	Your existing VM account and private key on your workstation
Administrator network	Your institutional VPN or stable public address as a CIDR
Source repository	`https://github.com/davidulicio/carbonique-webapp.git`
Deployment account	An existing human SSH account with sudo
Application account	A new restricted Linux account named `carbonique`
Application port	`127.0.0.1:8010`, reserved for this installation
Email service	Approved SMTP host, port, sender and credentials
Backup destination	A private copy on a different system, with a tested restore procedure

The hostname must point to the existing floating IP. Ask your institution's DNS administrator to create its A record. Add an AAAA record only if IPv6 actually works and is protected. A dedicated hostname avoids conflicts with other websites; serving this app under a subfolder is unsupported because its API and service worker use paths from `/`.

2 Connect and inspect the existing server

On your Windows workstation, open PowerShell. Substitute your own values:

```
ssh -i "$env:USERPROFILE\.ssh\YOUR_KEY" YOUR_SSH_USER@YOUR_FLOATING_IP
```

Verify the SSH host-key fingerprint with the VM administrator the first time. Do not disable host-key verification. Once connected, the commands below run in the VM's Linux shell, unless a step explicitly says Windows.

```
whoami
sudo -v
cat /etc/os-release
free -h
df -hT
findmnt /Data
lsblk -o NAME,SIZE,FSTYPE,MOUNTPOINTS
sudo ss -lntup
systemctl --type=service --state=running
sudo ufw status verbose
```

Identify who owns ports 80 and 443, whether tower collection jobs run here, and how `/Data` is mounted. Inspect existing scheduled jobs with the administrator, including systemd timers and the relevant users' crontabs. Keep this inventory private because it can expose operational details.

If nginx owns the web ports, use a new named virtual host in step 12. If Apache owns them, use the Apache alternative in step 13. If another proxy or application owns them, have its administrator add a route for the new hostname before continuing. Do not stop an existing web server or replace its configuration.

3 Protect tower data and plan storage

Hosting here is feasible for a small team, subject to observed workloads. Start with two Gunicorn workers and monitor resource use. Keep tower acquisition and processing jobs under their existing accounts. The app account should not own or have write access to raw tower data. The app currently does not need `/Data` to run field records, public notes or its reference pages.

The large `/Data` mount is network storage, not extra space on the system disk. This installation keeps PostgreSQL on a local or attached persistent block filesystem and private uploads in `/var/lib/carbonique/media`. Do not move the database onto this NFS mount as a shortcut. Plan a separate persistent block volume for growing uploads/database storage with the cloud administrator. Do not format `/dev/vda`, the boot partitions or any existing data device.

Set a local operational alert before the root disk reaches 80 percent usage. Allow space for the Python environment, packages, database growth, temporary uploads and backups. A full backup can temporarily require another copy of all uploaded files. The 20 GB disk is a starting constraint, not a long-term media archive. A larger shared mount does not establish that your app data is backed up.

4 Tighten Arbutus network access

Use the Arbutus dashboard associated with your project. In Horizon, select the project, then **Network > Floating IPs** and confirm that the existing IP is associated with this instance. Do not release or disassociate it. The supplied quota message concerns allocating another IP, which this deployment does not need.

Under **Network > Security Groups**, create a group for this VM rather than editing a shared default group used by other instances. Add these inbound rules:

Protocol and port	Source	Purpose
TCP 22	Approved administrator/VPN CIDR	SSH maintenance

Protocol and port	Source	Purpose
TCP 80	`0.0.0.0/0`	Certificate validation and redirect to HTTPS
TCP 443	`0.0.0.0/0`	Public report and authenticated app
Existing research services	Only their approved sources	Preserve documented acquisition, storage or management traffic

If IPv6 is used, add the equivalent intended rules using appropriate IPv6 CIDRs. Leave database, Gunicorn and tower device ports closed to the public internet. The app needs no inbound UDP rule. This is separate from UDP required by other existing services, which the administrator must assess.

The screenshots show rules allowing all inbound IPv4 TCP and UDP traffic. A new restrictive group does not cancel permissive rules from another attached group: OpenStack combines allowed traffic. First attach the replacement group through **Compute > Instances > Edit Security Groups**. Verify a new SSH connection, existing data collection and the storage mount. Keep the original SSH session and recovery console available. Only then detach the permissive group from this instance, after its administrator confirms all required rules are retained. Do not delete a shared security group or remove every rule in bulk.

Outbound rules marked **to** differ from inbound rules marked **from**. Retain the existing egress required for DNS, time, package updates, cloud metadata, SMTP, NFS and tower connections. Restrict it later only with a complete network inventory. Do not publish rpcbind, NFS, SMTP or unexplained high ports merely because a process is listening on them.

5 Check the guest firewall without losing access

The cloud security group and the VM's firewall are separate controls. If a host firewall is already managed, extend that policy with its administrator. Do not reset UFW or install competing firewall rules over a managed policy.

For a VM where the administrator has chosen UFW and confirmed the existing service inventory, add the approved rules before enabling it. Replace the documentation CIDR below with the network from which you are actually connecting.

```
sudo ufw allow proto tcp from 198.51.100.0/24 to any port 22
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw status numbered
```

Add any necessary source-restricted research service rules as agreed in step 4. Then enable UFW if it was previously inactive, verify a fresh SSH connection and check data collection. Do not close the original connection until this succeeds.

```
sudo ufw enable
sudo ufw status verbose
```

The app never requires opening TCP 8010 or 5432 in UFW or Arbutus. Existing broad allow rules must also be reviewed; adding narrower rules alone does not remove them.

6 Install the runtime and application account

Review and apply Ubuntu security updates with the administrator. Updates and a reboot can interrupt tower jobs, so agree on their timing first. Install the app dependencies without changing the active web server:

```
sudo apt update
sudo apt install python3-venv python3-pip git postgresql postgresql-client curl
```

Confirm that no existing account or application already uses the name `carbonique`. If it does, choose a separate service/account/database name and adjust all paths and configurations consistently. Do not change its ownership.

```
getent passwd carbonique
```

For a new account, run:

```
sudo adduser --system --group --home /var/lib/carbonique --no-create-home carbonique
sudo install -d -o carbonique -g carbonique -m 0750 /var/lib/carbonique/media
sudo install -d -o root -g carbonique -m 0750 /etc/carbonique
sudo install -d -o carbonique -g carbonique -m 0700 /var/backups/carbonique
sudo install -d -o "$USER" -g carbonique -m 2750 /srv/carbonique-webapp
```

The human deployment account owns the source directory. The service account reads the code and writes private uploads. Do not give it sudo rights or a login password. Do not recursively change ownership or permissions on `/Data`.

7 Install the reviewed source

The destination must be empty on the first installation. Use the reviewed repository, after your local changes have been pushed there:

```
umask 0027
git clone https://github.com/davidulicio/carbonique-webapp.git /srv/carbonique-webapp
cd /srv/carbonique-webapp
git status --short
git log -1 --oneline
python3 -m venv .venv
.venv/bin/python -m pip install --upgrade pip
.venv/bin/python -m pip install -r requirements.txt
sudo install -d -o carbonique -g carbonique -m 0750 /srv/carbonique-webapp/staticfiles
```

Record the deployed commit in your maintenance record. If the repository is private, use an approved read-only deploy key or interactive GitHub access workflow. Never put a token into a clone URL or commit it. The VM needs no Node build step. The app and protocol files in Git are distinct from your current local database and uploaded media, which must be migrated in step 10 if needed.

8 Create a dedicated PostgreSQL database

Inspect PostgreSQL first so that an existing cluster is not overwritten:

```
pg_lsclusters
sudo -u postgres psql -c '\du'
sudo -u postgres psql -c '\l'
```

For a new role/database named `carbonique` on the default local cluster:

```
sudo -u postgres createuser --no-superuser --no-createdb --no-createrole carbonique
sudo -u postgres createdb --owner=carbonique carbonique
sudo -u carbonique psql -h /var/run/postgresql -d carbonique -c 'SELECT current_user, current_database();'
```

The final command should show `carbonique` for both values. This guide uses local Unix socket peer authentication: the Linux service account matches the PostgreSQL role, and the web app needs no database password. Confirm the local authentication policy with the administrator if that check fails. Do not change authentication to `trust`. If your existing cluster uses another port, use that port in these commands and `DB_PORT` below.

```
sudo -u postgres psql -c 'SHOW listen_addresses;'
sudo -u postgres psql -c 'SHOW data_directory;'
```

Keep the database private. Do not change a shared PostgreSQL instance's listening or authentication policy without assessing its other databases.

9 Configure private production settings

Create the settings file outside the checkout and make a symlink for Django's existing dotenv loader. Check that the checkout has no `.env` before creating the link; preserve an existing configuration if this is an upgrade.

```
sudo touch /etc/carbonique/app.env
sudo chown root:carbonique /etc/carbonique/app.env
sudo chmod 0640 /etc/carbonique/app.env
ls -la /srv/carbonique-webapp/.env
sudo ln -s /etc/carbonique/app.env /srv/carbonique-webapp/.env
sudoedit /etc/carbonique/app.env
```

An absent `.env` at the `ls` check is expected on a fresh clone. Paste this configuration, substituting your approved hostname and email settings:

```
DEBUG=False
SECRET_KEY=REPLACE_WITH_A_NEW_RANDOM_VALUE
ALLOWED_HOSTS=carbonique.example.org,localhost,127.0.0.1
CSRF_TRUSTED_ORIGINS=https://carbonique.example.org
DATABASE_URL=
DB_ENGINE=django.db.backends.postgresql
DB_NAME=carbonique
DB_USER=carbonique
DB_PASSWORD=
DB_HOST=/var/run/postgresql
DB_PORT=5432
MEDIA_ROOT=/var/lib/carbonique/media
PUBLIC_FIELDNOTES_ENABLED=True
TRUST_PROXY_HEADERS=True
TRUSTED_PROXY_NETWORKS=127.0.0.1/32,::1/128
SECURE_SSL_REDIRECT=True
EMAIL_BACKEND=django.core.mail.backends.smtp.EmailBackend
EMAIL_HOST=YOUR_APPROVED_SMTP_HOST
EMAIL_PORT=587
EMAIL_USE_TLS=True
EMAIL_HOST_USER=YOUR_SMTP_ACCOUNT
EMAIL_HOST_PASSWORD=YOUR_SMTP_SECRET
DEFAULT_FROM_EMAIL=YOUR_APPROVED_SENDER
MONITOR_STATIONS_FILE=
```

Generate the secret privately and paste it into `SECRET_KEY`. Do not send it in a support message or put it in Git. Keep at least 50 random characters:

```
.venv/bin/python -c 'import secrets; print(secrets.token_urlsafe(64))'
```

The empty `DATABASE_URL` is intentional because the explicit PostgreSQL settings above are used instead. Do not copy the local development console email backend into production. Port 587 with STARTTLS is an example; use the options supported by your approved SMTP service. The current settings expose a STARTTLS option, not a separate implicit SSL option for port 465.

Proxy headers are trusted here only because Gunicorn binds to loopback and the front proxy overwrites them. Do not expose the backend publicly or trust an arbitrary client-supplied IP header. No GitHub token is required for the INI link: users open the upstream repository directly.

10 Initialize or migrate your current records

Run migrations in the new database first:

```
cd /srv/carbonique-webapp
sudo -u carbonique .venv/bin/python manage.py migrate
```

Choose exactly one starting path.

A Fresh installation from bundled references. This creates the supplied sites, inventory, visit summaries, caveats, scientific evidence, calibration history and seven protocols. It does not import an old EcoFlux database:

```
sudo -u carbonique .venv/bin/python manage.py setup_hub --with-reference
sudo -u carbonique .venv/bin/python manage.py createsuperuser
```

Use a unique password of at least 12 characters. There are no default accounts. Then create your team in Administration and assign the least role/site access each person needs.

B Move the records you have already edited locally. Before taking the backup, stop local writers and synchronize every device's pending field queue. Keep the local app closed to edits until cutover is complete. In Windows PowerShell, in your local `carbonique-webapp` directory:

```
.\.venv\Scripts\python.exe manage.py backup_hub --output backups\server-transfer.zip
scp -i "$env:USERPROFILE\.ssh\YOUR_KEY" .\backups\server-transfer.zip
    YOUR_SSH_USER@YOUR_FLOATING_IP:~/server-transfer.zip
```

The archive includes account password hashes, database records and uploaded media. Keep it private. Back up the private environment and monitor configuration separately; they are not included. It also excludes unsynchronized device queues.

On the VM, install the uploaded archive into private storage and restore it:

```
chmod 600 ~/server-transfer.zip
sudo install -o carbonique -g carbonique -m 0600 ~/server-transfer.zip
    /var/backups/carbonique/server-transfer.zip
sudo -u carbonique .venv/bin/python manage.py restore_hub --file /var/backups/carbonique/server-transfer.zip
```

Restore requires a migrated, empty database with no users/sites and an empty media directory. Do not run setup or createsuperuser before this path. The restore command refuses an occupied target; do not bypass that protection. Use a new target if a first attempt needs investigation. The portable restore limit is 20 GB uncompressed; use administrator-managed database/media migration for larger archives.

Restoring an older backup can restore its earlier site display names. When needed, run `sudo -u carbonique .venv/bin/python manage.py update_site_names` after verifying the restore. This deliberately applies the six approved names. Other imported source material must not overwrite later team edits.

For both paths, finish with:

```
sudo -u carbonique .venv/bin/python manage.py collectstatic --noinput
sudo -u carbonique .venv/bin/python manage.py check --deploy
```

Resolve errors and unexpected warnings. `security.W021` is expected because HSTS preload is intentionally disabled. This does not justify ignoring any other finding. Compare site, visit, calibration and caveat counts and inspect attachments before allowing new writes.

11 Run Gunicorn as a managed service

Confirm port 8010 is free. If it is occupied, select another free loopback port and change both the service and proxy examples to match it:

```
sudo ss -lntp 'sport = :8010'
sudo cp deploy/arbutus.service.example /etc/systemd/system/carbonique.service
sudo systemctl daemon-reload
sudo systemctl enable --now carbonique
sudo systemctl status carbonique --no-pager
```

The supplied service uses two workers, a 120 second timeout, a dedicated account and filesystem restrictions. It has write access only to private app uploads plus its isolated temporary directory. It does not grant access to tower data. Inspect the private logs if it does not start:

```
sudo journalctl -u carbonique -n 80 --no-pager
curl -fsS -H 'Host: carbonique.example.org' -H 'X-Forwarded-Proto: https' http://127.0.0.1:8010/health/
```

Expect an HTTP 200 JSON response with status `ok`. The host and HTTPS headers are intentional for this loopback-only test with production HTTPS settings. Do not run Django `runserver` as the production service.

12 Add a hostname to an existing nginx server

Use this path if nginx already owns ports 80/443, or if neither port is occupied and the administrator chooses nginx. In the latter case install it with `sudo apt install nginx`. On a shared server, preserve all existing sites and default servers. Back up the nginx configuration privately first:

```
sudo cp -a /etc/nginx /root/nginx-before-carbonique
sudo nginx -t
```

Create an HTTP-only hostname configuration first, so nginx does not look for a certificate that does not exist yet. Replace the hostname throughout:

```
sudo install -d -o root -g root -m 0755 /var/www/carbonique-acme
sudoedit /etc/nginx/sites-available/carbonique
```

```
server {
    listen 80;
    server_name carbonique.example.org;
    location ^~ /.well-known/acme-challenge/ {
        root /var/www/carbonique-acme;
        try_files $uri =404;
    }
    location / { return 301 https://carbonique.example.org$request_uri; }
}
```

Enable only this new site, check syntax and reload. Do not replace nginx.conf or delete another site's configuration:

```
sudo ln -s /etc/nginx/sites-available/carbonique /etc/nginx/sites-enabled/carbonique
sudo nginx -t
sudo systemctl reload nginx
```

Install Certbot if this VM does not already have a managed certificate tool. Use one installation method rather than mixing apt, snap and pip:

```
sudo apt install certbot
sudo certbot certonly --webroot -w /var/www/carbonique-acme -d carbonique.example.org
```

Provide the administrator's email and review the certificate provider's terms. DNS must resolve correctly and TCP 80 must reach this hostname. If IPv6 DNS exists, it must also work. Use the existing institutional certificate process instead if required. Do not use Certbot standalone mode against occupied web ports.

After the certificate exists, copy the hostname configuration and edit it:

```
sudo cp deploy/arbutus-nginx.conf.example /etc/nginx/sites-available/carbonique
sudoedit /etc/nginx/sites-available/carbonique
sudo nginx -t
sudo systemctl reload nginx
```

Replace every `carbonique.example.org` and the sample administrator CIDR. The template includes only this app's named servers, keeps ACME validation working, restricts `/admin/`, adds request limits and sanitized access logging, and proxies to loopback port 8010. It deliberately does not introduce default servers that could replace existing websites. Include this file only once inside nginx's `http` context, as Ubuntu's sites-enabled layout normally does.

Check that these uniquely named rate-limit zones/log maps do not already exist before installing a duplicate copy. Keep the original general nginx template for a dedicated-server deployment; do not install both examples unchanged.

```
curl -I https://carbonique.example.org/accounts/login/
curl -fsS https://carbonique.example.org/health/
sudo certbot renew --dry-run
systemctl list-timers --all
```

Ensure a renewal timer is enabled for your chosen Certbot installation. The webroot method needs nginx reloaded after a successful renewal; create a root owned executable deploy hook under `/etc/letsencrypt/renewal-hooks/deploy/` that runs `/usr/sbin/nginx -t && /bin/systemctl reload nginx`. Confirm the hook and renewal job with the VM administrator. Do not assume a certificate renews simply because the first request succeeded.

13 Apache alternative when Apache already owns HTTPS

Do not install a second server on ports 80/443. Keep Apache as the front proxy and ask its administrator to add a separate virtual host for this hostname. Gunicorn, PostgreSQL and the initialization steps remain the same. Enable the required modules with the administrator, leaving existing virtual hosts intact:

```
sudo a2enmod proxy proxy_http headers ssl
sudo apache2ctl configtest
```

The HTTPS virtual host needs the correct `ServerName`, its existing managed certificate paths, `ProxyRequests Off`, `ProxyPreserveHost On`, `ProxyAddHeaders Off`, a proxy to `http://127.0.0.1:8010/`, and overwritten forwarding headers. For example, inside that hostname's HTTPS virtual host:

```
ProxyRequests Off
ProxyPreserveHost On
ProxyAddHeaders Off
RequestHeader set X-Forwarded-Proto "https"
RequestHeader set X-Real-IP "expr=%{REMOTE_ADDR}"
RequestHeader unset X-Forwarded-For
ProxyPass / http://127.0.0.1:8010/ connectiontimeout=5 timeout=120
ProxyPassReverse / http://127.0.0.1:8010/
LimitRequestBody 110100480
<Location /admin/>
    Require ip 198.51.100.0/24
</Location>
```

Replace the administrator CIDR. Use Apache's existing certificate process or Certbot webroot configuration, and provide HTTP ACME validation before forcing an HTTPS redirect. Add site-specific bot/request limiting with the existing approved WAF/proxy policy; nginx rate-limit directives cannot be copied into Apache. Configure access logs so password-reset token paths and query strings are not recorded. Deny direct filesystem access to the repository and media. Check syntax before reloading Apache and test existing websites afterward. If those controls are not in place, keep the deployment private until the server administrator finishes this proxy integration.

14 Configure tower status privately

Field records do not require live tower connections. Enable diagnostics only when the VM has the approved private route or VPN to the instruments. Follow Resources and monitoring for the JSON schema and the example under `deploy/`. Store the real file under `/etc/carbonique/monitor-stations.json`, owned by root, group carbonique, mode

0640. Set `MONITOR_STATIONS_FILE` to that absolute path in app.env, then restart

the service.

Use site codes to associate connections, not their display names. The current app uses restricted read-only TCP diagnostics. It does not require LI-COR web or SSH administrator passwords. Never expose analyzer ports through the public security group and never put actual tower addresses or credentials in Git. Test one tower, inspect the check time/data age, then test all towers.

15 Verify the application and phone experience

Use the real HTTPS hostname from an ordinary browser and from a phone on mobile data. Check both portrait and landscape. The internal app has a collapsible phone menu, wrapping actions, single-column forms and local scrolling for wide tables/tabs. There is no separate phone installation package.

1. Without signing in, open `/fieldnotes/`. Check the six site names, descriptions, conditions, visits, caveats and the timeline. Follow the INI link to GitHub.
2. Confirm that `/`, private APIs and private attachments require an authorized account. Verify another site's restricted records cannot be read by a user who lacks access. A login redirect or permission response is expected.
3. Sign in with a test team account. Open every workspace section and select each site. Open/close the phone menu, a form, a file preview and a timeline marker.
4. Create a clearly labeled test visit/note and attach a harmless PDF/image. Record a test calibration. Check downloads and viewer/manager role behavior.
5. Add a test caveat, review it and mark it resolved. Reload the public page and confirm both the record and timeline change. Check site-condition edits too.
6. Test the SMTP password-reset flow with a test account, including receipt of the message and a working HTTPS link. Keep that link and logs private.
7. On the phone while online, use **Prepare for fieldwork** for a test site. Switch the device offline, reopen the prepared workspace and record a test note. Reconnect, keep the app open, wait for the queue to reach zero and verify the server record. The GitHub INI link requires internet access.
8. Confirm `/admin/` works only from its approved network. Check that other existing websites, the `/Data` mount and tower collection still work.

Use controlled test records and archive them through the normal app workflow where available. Do not use real production accounts to repeatedly test lockout. The app limits five failed account/client sign-ins for 15 minutes; the front proxy adds request limits. These controls reduce abuse but cannot guarantee that no future vulnerability will exist.

From Windows, confirm private application/database ports are unreachable from the public network. Replace the IP and repeat for each port:

```
Test-NetConnection YOUR_FLOATING_IP -Port 8010
Test-NetConnection YOUR_FLOATING_IP -Port 5432
```

Expect `TcpTestSucceeded : False`. Verify HTTPS separately on port 443. Check that the result is from the intended network and that private VPN routing is not changing the test.

16 Back up and practice a restore

Assign a backup owner and retention policy before allowing routine writes. Back up database records, private media and private settings. Keep at least one encrypted/private copy off this VM. A snapshot alone does not cover external NFS data or every attached volume. Coordinate snapshots with the storage owner.

For a consistent portable app backup, pause writers, including staff using the app. Stop only the app service, not the tower collection or existing web server:

```
sudo systemctl stop carbonique
cd /srv/carbonique-webapp
sudo -u carbonique .venv/bin/python manage.py backup_hub --output
    /var/backups/carbonique/REPLACE_WITH_UNIQUE_DATE.zip
sudo systemctl start carbonique
```

Do not reuse a backup filename. Keep the app stopped until a failed backup is investigated or an alternative backup is confirmed. Restarting after a failure is an explicit operational decision. For larger production systems, add native PostgreSQL backups and a coordinated media snapshot with the administrator.

Restore into a separate empty database/media directory or a separate test VM; never test a restore against the live target. Follow path B in step 10 and verify counts, login, files and public output. Record the successful restore date. Backups do not include browser queues, so synchronize devices first.

17 Update and recover a release

Before an update, record the current commit, synchronize devices, pause writers, make a verified backup and confirm free disk space. Keep the deployment account and maintenance window available. Stop the app service, then run as the source owner from the repository directory:

```
git status --short
git log -1 --oneline
git pull --ff-only
.venv/bin/python -m pip install -r requirements.txt
sudo -u carbonique .venv/bin/python manage.py migrate
sudo -u carbonique .venv/bin/python manage.py collectstatic --noinput
sudo -u carbonique .venv/bin/python manage.py check --deploy
sudo systemctl start carbonique
```

If Git shows unexpected changes or a command fails, stop and investigate rather than resetting the checkout. Do not automatically reimport reference data over team edits. Check the release notes before running any import command. Changes saved in the app reach the public site on its next refresh; a new code release is not needed for each edit. GitHub maintains current INIs separately.

After updating, repeat health, login, public record, phone and attachment checks. Open the workspace online so the new service worker/assets can load, then prepare the field device before disconnecting. Pending device records are not intentionally erased by an asset update.

If a release fails after a schema change, keep writers paused. A code rollback alone may not match the changed database. Have the administrator restore the matching database/media backup into a new target and point the app to it, or follow a tested release-specific rollback. Preserve the failed target for diagnosis.

18 Troubleshoot common problems

Symptom	Check and next action
SSH stops working	Use the instance console; confirm the current source CIDR, attached security groups, UFW and key. Do not widen every port.
Existing website becomes unavailable	Revert only the new virtual host, test proxy syntax and reload. Check hostname/default-server conflicts.
502 from the proxy	Check systemctl status, private journal, port 8010 and the loopback health request.

Symptom	Check and next action
Repeated HTTPS redirect	Confirm the proxy overwrites X-Forwarded-Proto and TRUST_PROXY_HEADERS is true. Never fix this by leaving production HTTP enabled.
400 Invalid host	Match ALLOWED_HOSTS to the real hostname; do not use a wildcard.
403 when saving a form	Check the HTTPS origin, CSRF settings, session and cookies. Keep CSRF protection enabled.
429 during login	Wait for the lockout/cooldown. Queued records remain on the device; review the proxy limit if a shared VPN affects many users.
PostgreSQL peer authentication fails	Confirm the Linux user is carbonique, the matching DB role exists, the socket/port is correct and local pg_hba rules permit peer.
Permission denied for a PDF/upload	Check MEDIA_ROOT ownership, the service ReadWritePaths setting, disk space and site access. Never use chmod 777.
Phone has old styles	Open online and refresh the app, then prepare again. Do not clear storage with pending records.
Public output seems old	Upload pending edits and refresh `/fieldnotes/`; check that the correct hostname/database is in use.
INI link needs sign-in or is unavailable	Check repository permissions in GitHub and the device's internet connection. The app holds no GitHub token.
Tower unreachable	Check private routing/VPN, the private station JSON, site code and device port. Do not expose the instrument publicly.
Disk space falls rapidly	Check private media, journals, database and local backups. Move retained backups off the root disk under the retention policy.

Keep journal output private; it can contain operational details. The configured nginx access log omits query strings and redacts password-reset token paths.

19 Plan future tower data plots

A plotting section can be added later on the same VM. Start with read-only access to a dedicated directory or derived data export. Keep raw acquisition files outside web-accessible paths, and grant access only to selected datasets. Do not make `/Data` generally readable to the app or mount it as public media.

Use bounded date/variable queries, validated file formats, caching and downsampling. Long processing should run separately with CPU/memory limits so phone charts and web requests do not compete with tower acquisition. Decide which plots are public before implementing an API. If web traffic or processing grows, move the app/plot worker to a separate VM while leaving raw data collection isolated. More RAM alone does not solve disk growth or data-access risk.

20 Documentation and handover

Record the hostname, deployed commit, service/database owners, storage mount locations, backup/restore results, certificate renewal, SMTP owner, network rules and the emergency contact in private operational documentation. Keep passwords, private keys and tokens in the approved secret store.

Official references for the administrator, checked 21 October 2026:

- Alliance technical documentation and Cloud Quick Start
- Alliance account application, for a new colleague who needs an account, not a prerequisite to repeat for an existing account
- OpenStack instance security and floating IPs
- Ubuntu firewall administration
- Ubuntu nginx configuration
- Ubuntu PostgreSQL installation
- Django deployment checklist
- Certbot webroot and renewal
- Apache reverse proxy
- Apache RequestHeader

Alliance cloud policies and dashboard details can change. Consult the project administrator and those official pages for current allocation, access and cloud specific requirements. The operational commands above apply to the existing Ubuntu VM and the source configuration described in this guide.